communityclient 插件深度审查报告

> 版本：1.0.0 | 类型：前端插件 | 代码量：~900 行
>
> 审查日期：2026-04-03

综合评分

维度	评分	简评
功能性	5/10	API 层完整，但大量后端功能无对应 UI
安全性	4/10	Token 明文存储，无输入长度限制
架构设计	7/10	GetX 分层清晰，DI 设计合理
正确性	4/10	点赞数显示 Bug、评论用户名未使用、路由参数缺失等
错误处理	2/10	全部静默失败，用户看不到任何错误提示
可测试性	2/10	Controller 强耦合单例 API 服务，无法 Mock
总评	**4/10**	框架搭得好，但正确性与用户体验有明显短板

一、功能性分析

API 层 vs UI 层覆盖对比

功能	API 方法 ✅	Controller 🔄	UI 实现 ✅
获取文章列表	✅	✅	✅
创建/编辑文章	✅	✅	✅
删除文章	✅	❌	❌
文章详情	✅	✅	✅（简陋）
评论列表	✅	✅	✅
发表评论	✅	✅	✅
删除评论	✅	❌	❌
评论点赞	❌	❌	❌
文章点赞	✅	✅	✅
关注用户	✅	❌	❌
举报文章	✅	❌	❌
不喜欢文章	✅	❌	❌
阅读标记	❌	❌	❌
媒体上传	❌	❌	❌
黑名单	❌	❌	❌
屏蔽分类	❌	❌	❌
分类选择	✅（API）	❌	❌
我的草稿	❌	❌	❌
我的文章	❌	❌	❌

API 层对后端已实现的大部分接口做了封装，但只有不到一半的功能走通了 **Controller → UI** 这条链。

二、Bug 批判

🔴 Bug 1：文章列表将媒体数量误显示为点赞数

```
// articles_list_view.dart  line 144
Row(
  children: [
    const Icon(Icons.thumb_up_outlined, size: 14, color: Colors.grey),
    const SizedBox(width: 2),
    Text('${article.media.length}',  // ← 点赞图标，却显示媒体数量
        style: theme.textTheme.labelSmall),
  ],
),
```

`Article` proto 中有 `repeated Media media` 字段，用 `media.length` 作为点赞数完全错误。正确做法应从 `getArticleDetail` 的 `likelist.length` 获取，或后端在 Article 上冗余一个 `likeCount` 字段。

快速修复：当前 `GetArticlesResponse` 中的 `Article` 没有 likeCount，短期内可以隐藏点赞数或请求后端在 proto 中补充字段。

🔴 Bug 2：评论展示忽略接口已返回的用户名

```
// article_detail_view.dart
class _CommentTile extends StatelessWidget {
  final String content;
  final int userId;

  @override
  Widget build(BuildContext context) {
    // ...
    Text('用户 $userId',   // ← 硬编码拼接，即使接口已返回 userName
        style: Theme.of(context).textTheme.labelSmall),
```

后端 `GetCommentsResponse.Comment` 中已包含 `userName` 和 `userImageUrl`，前端完全忽略，改为显示 `用户 123` 这样的占位文本。用户体验很差。

🟠 Bug 3：文章详情路由的 `userId` 丢失

```
// articles_list_view.dart — ArticleCard.onTap
Get.toNamed(
  CommunityRoutes.articleDetail,
  arguments: {'articleId': article.articleID},
  // 没有传 userId！
);
```

`ArticleDetailView` 中从 `arguments` 读取 `userId`，默认值为 `0`：

```
final int userId = args['userId'] as int? ?? 0;
```

`userId = 0` 导致：

- 无法正确判断当前用户是否已点赞（`isLiked` 永远为 false）
- 无法显示文章作者信息（`getArticleDetail` 以 userID=0 查询）
- 无法判断是否显示编辑按钮（`a.userID == userId` → `0 == userId` 永远为 false）

🟠 Bug 4：`_ensureInit` 的重初始化逻辑是空操作

```
// community_api_service.dart
void _ensureInit() {
  if (_initialized) return;
  _dio = Dio(BaseOptions(baseUrl: _baseUrl, ...));
  // ...
  _initialized = false; // defer re-init when baseUrl changes
  _initialized = true;
}
```

注释写"baseUrl 变化时 defer re-init"，但代码逻辑是：先设 `false`，立即设 `true`，完全等价于直接设 `true`。当 `baseUrl` 发生变化（如环境切换），Dio 实例永远不会重建，新的 baseUrl 不会生效。

🟡 Bug 5：`ArticleDetailView` 有条件懒注册 Controller

```
// article_detail_view.dart
if (!Get.isRegistered<ArticleDetailController>()) {
  Get.put(ArticleDetailController(articleId: articleId, userId: userId));
}
```

View 中做注册是反模式，且检查 `isRegistered` 带来副作用：从文章 A 进入详情，再返回，再进入文章 B，`ArticleDetailController` 已注册，懒注册被跳过，显示的仍是文章 A 的数据。

三、安全性批判

🟠 Token 明文存储于 SharedPreferences

```
// community_api_service.dart
final prefs = await SharedPreferences.getInstance();
final token = prefs.getString('accessToken') ?? '';
```

`SharedPreferences` 在 Android 上以明文 XML 存储，Root 用户或备份攻击可直接读取 access token。更安全的做法是使用 `flutter_secure_storage`（底层 Android Keystore / iOS Keychain）。

🟡 评论输入无长度限制

```
// article_detail_view.dart
TextField(
  controller: _commentCtrl,
  decoration: const InputDecoration(hintText: '写评论…'),
  // 无 maxLength，无 maxLines 截断
)
```

用户可以提交任意长度评论，后端也没有对 `content` 做长度校验，超长内容会直接插入数据库。

四、架构设计批判

问题 1：Controller 强耦合 `CommunityApiService` 单例，无法 Mock

```
class CommunityHomeController extends GetxController {
  final _api = CommunityApiService();  // 直接拿单例，无法注入
```

所有 Controller 直接引用 `CommunityApiService()` 单例工厂，无法在测试中替换为 Mock 实现，这也是为什么 communityclient 没有任何测试。

改良方案：通过构造函数注入：

```
class CommunityHomeController extends GetxController {
  final CommunityApiService _api;
  CommunityHomeController({required this.userId, CommunityApiService? api})
    : _api = api ?? CommunityApiService();
```

问题 2：proto 枚举值以魔数方式硬编码

```
// community_home_controller.dart
// ArticleClassification (从 community.proto): discover=2, recommended=1,
follow=3
static const int _clsDiscover    = 2;
static const int _clsRecommended = 1;
static const int _clsFollow      = 3;
```

注释本身证明这些是 proto 枚举的数值，应该直接用 `ArticleClassification.discover.value` 等（proto 生成的 Dart 代码中枚举有 `.value` 属性），避免与后端枚举定义脱钩。

问题 3：`articleCreate` Binding 中丢失 `existingArticle`

```
// community_routes.dart
GetPage(
  name: articleEdit,
  page: () => const ArticleCreateView(),
  binding: ArticleCreateBinding(userId: userId),  // 没有传 existingArticle
),
```

`ArticleCreateBinding` 注入的 `ArticleCreateController` `existingArticle` 永远为 null，编辑文章时无法加载已有内容到编辑器。

五、错误处理批判

全部 IO 操作静默吞掉异常

```
// 三个 Controller 中重复同样的模式
Future<void> loadDiscover({bool refresh = false}) async {
  // ...
  try {
    final resp = await _api.getArticles(req);
    // ...
  } catch (e) {
    _log.warning('loadDiscover error: $e');  // 仅记日志，界面无任何反馈
  }
}
```

网络断开、服务器 500、token 过期……所有错误的处理方式均为"打个 warning 日志，假装一切正常"。用户看到的是永久转圈或空白页面，没有任何错误提示、重试按钮或 fallback UI。

改良方案：引入错误状态并在 UI 中响应：

```
final errorMsg = Rxn<String>();

} catch (e) {
  _log.warning('loadDiscover error: $e');
  errorMsg.value = '加载失败，请检查网络';
}

// UI
Obx(() {
  if (ctrl.errorMsg.value != null) {
    return ErrorView(
      message: ctrl.errorMsg.value!,
      onRetry: () => ctrl.loadDiscover(refresh: true),
    );
  }
  // ...
})
```

六、改良建议总结

优先级	改良项	预估工作量
P0	修复 ArticleCard 点赞数显示（`media.length` → 实际 likeCount）	1h
P0	修复 `articles_list_view` 跳转详情时不传 `userId`	0.5h
P0	修复 `ArticleDetailController` 懒注册导致切换文章数据不更新	1h
P0	评论展示使用接口返回的 `userName` / `userImageUrl`	1h
P0	修复编辑文章路由不传 `existingArticle`	1h
P1	全部 catch 块补充 UI 错误状态和重试机制	4h
P1	Token 改用 `flutter_secure_storage` 存储	2h
P1	Controller 构造函数注入 API 服务，解除单例耦合	2h
P1	修复 `_ensureInit` 无效的 re-init 逻辑	0.5h
P2	用 proto 枚举值替代魔数常量	0.5h
P2	评论输入框加 `maxLength` 和服务端同步长度校验	1h
P3	实现删除文章 / 删除评论 UI	3h
P3	实现关注用户 UI	2h
P3	实现 "我的文章 / 草稿" 列表	4h